# Amalgam: Hybrid LLM-PGM Synthesis Algorithm for Accuracy and Realism

**Antheas Kapenekakis** [1] **Bent Thomsen** [1] **Katja Hose** [2] **Michele Albano** [1]

## Abstract

To generate synthetic datasets, e.g., in domains such as healthcare, the literature proposes approaches of two main types: Probabilistic Graphical Models (PGMs) and Deep Learning models, such as LLMs. While PGMs produce synthetic data that can be used for advanced analytics, they do not support complex schemas and datasets. LLMs on the other hand, support complex schemas but produce skewed dataset distributions, which are less useful for advanced analytics. In this paper, we therefore present Amalgam, a hybrid LLM-PGM data synthesis algorithm supporting both advanced analytics, realism, and tangible privacy properties. We show that Amalgam synthesizes data with an average 91 % $\chi^2 P$ value and scores 3.8/5 for realism using our proposed metric, where state-of-the-art is 3.3 and real data is 4.7.

## 1. Introduction

Data synthesis is the process of generating artificial data that mimics the properties of a real dataset. It is widely used in multiple domains, with two main purposes: augmenting training data for machine learning and sharing data in a privacy-preserving manner. In this paper, we focus on the latter, which is especially relevant in domains that handle confidential data, such as healthcare and finance. In these domains, algorithms focus on two approaches: Probabilistic Graphical Models (PGMs) and Deep Learning Networks.

Probabilistic Graphical Models (PGMs) (Cai et al., 2023; 2021; McKenna et al., 2022) are low parameter statistical models that learn the joint distribution of a dataset. Because of their structure, they are interpretable and the resulting data has strong analytical properties (e.g., Kullback-Leiber divergence is minimal). In addition, due to their low parameter count, they have excellent privacy properties, especially when combined with methods such as Differential Privacy (DP) (Dwork, 2006), and through work such as the system Pasteur (Kapenekakis et al., 2026), they are shown to scale to very large datasets. However, PGMs do not scale to complex datasets, with most work focusing on tabular data (Cai et al., 2021; McKenna et al., 2022; Zhang et al., 2017; McKenna et al., 2021).

On the other hand, Deep Learning models (Jordon et al., 2018; Guo et al., 2023), especially Large Language Models (LLMs) (Goyal & Mahmoud, 2025; Chan et al.; Yang et al., 2024), due to their high parameter count and being pre-trained on a large corpus of data, have the potential to handle complex datasets, which may contain sequences or complex relational structures. However, due to their high parameter count, if the input dataset is used for training, it is hard to ensure that the model does not expose the training dataset and that it is privacy-preserving without sacrificing accuracy (Song et al., 2013). Moreover, the resulting data tends to capture a less accurate distribution (Tao et al., 2021)

These approaches appear complimentary: PGMs provide accurate analytics and privacy, while LLMs provide the ability to handle complex datasets. However, combining them and retaining these qualities remains an unanswered question. Evaluating the result for realism and analytics accuracy without a human in the loop remains an open question as well. In this paper, we aim to answer the following research questions:

**(RQ1):** How do we combine a PGM and an LLM to produce synthetic data where the following are met:

- **(Sub-RQ1):** We preserve the accuracy of analytics from the PGM? (accuracy)
- **(Sub-RQ2):** The foundational knowledge of an LLM is used to handle and enhance complex dataset? (complexity)
- **(Sub-RQ3):** The resulting data features quantified privacy properties? (privacy)
- **(Sub-RQ4):** The computational complexity remains reasonable to perform locally? (efficiency)

**(RQ2):** Given a relational dataset as a result, how do we evaluate the synthetic data for:

- **(Sub-RQ1):** Realism, i.e., how lifelike is the data?, in an unattended fashion, i.e., without review by a

[1]Aalborg University, Aalborg, Denmark [2]TU Wien, Vienna, Austria. Correspondence to: Antheas Kapenekakis <antheas@cs.aau.dk>.

*Preprint. March 27, 2026.*

human (realism)

(**Sub-RQ2**): Accuracy of analytics, i.e., how well key statistics are preserved?, across tables in a standardized manner (accuracy)

In this paper, we aim to combine the benefits of PGMs and LLMs, through introducing Amalgam, a hybrid LLM-PGM synthesis algorithm. Amalgam uses a PGM to learn the joint distribution of the dataset and generate summary statistics which are privacy-preserving and accurate. The summary statistics are used to condition an LLM, along similar samples from the original dataset, to generate synthetic samples in a zero-shot manner. The resulting data features a favorable combination of the qualities of privacy-preservation, accuracy, complexity of input data, and computational efficiency over state-of-the-art. Then, we present an unattended evaluation method for dataset realism, where an LLM is used to reason and then score how lifelike each sample is. Finally, we present experiments where we compare Amalgam against state-of-the-art on two public datasets for accuracy and realism. We show that Amalgam presents a favorable combination of qualities over state-of-the-art (i.e., higher privacy and accuracy than LLMs, handling higher complexity with higher realism than PGMs).

## 2. Related Work

Work on data synthesis spans multiple domains and data types, where foundation of this paper is relational data synthesis and privacy-aware synthesis. For structured data, most work focuses on tabular datasets and on two approaches. The first one is PGM models (McKenna et al., 2022; Cai et al., 2021; Zhang et al., 2017; McKenna et al., 2021; Mckenna et al., 2019), which feature the Differential Privacy technique (Dwork, 2006) to ensure privacy. The second one is Deep Learning models (Jordon et al., 2018; Guo et al., 2023; Xu et al., 2019). Most of these works involve the use of a Generative Adversarial Network (GAN) or a Variational Autoencoder (VAE). In benchmarks (Tao et al., 2021), PGM models tend to outperform Deep Learning models in terms of distributional accuracy on tabular datasets. Moreover, training Deep Learning models via Differential Privacy tends to result in a significant accuracy loss as shown in those papers (Jordon et al., 2018; Guo et al., 2023; Xu et al., 2019) and since tabular datasets are trivial, the ability of Deep Learning models to handle complex datasets is not leveraged.

Recently, two PGM works have attempted to synthesize relational data (Cai et al., 2023; Kapenekakis et al., 2024) using PGMs. PrivLava (Cai et al., 2023) derives from PrivMRF (Cai et al., 2021) to allow generating multiple disjoint tables together, without inter-table dependencies (e.g., two subsequent patient admissions in a medical dataset are generated independently). MARE (Kapenekakis et al., 2024) is PGM algorithm agnostic and models a relational dataset as a template representation graph (Koller & Friedman, 2009). This graph can model three types of child-to-parent table relationships: sequential (via Markov Chains), associative (via a technique called unrolling), and independent, where generated rows can reference all previously generated values (e.g., an admission column can reference the patient's sex). Both of these approaches are limited to simple relational datasets and scaling to more complex dataset requires significant engineering. This is similar to early attempts at computer vision, which used complex rulesets to e.g., recognize faces before being obsoleted by Deep Learning.

LLM models are used in general for generation purposes (Vaswani et al., 2017), and for synthetic data (Goyal & Mahmoud, 2025). Recently, great strides have been made on models that can run on-premises, in secure infrastructure (Lin et al., 2024; et al., 2024; GemmaTeam, 2025). These models can be effectively run on a single GPU with an adequate token generation speed (hundreds of tokens per second). However, small models are regarded as having a lower quality output than higher end proprietary models.

To aid in augmenting data quality, recent works suggest a grammar based approach (Ugare et al.; Tuccio et al., 2025)to limit the output tokens of the models to a specific schema (e.g., SQL, JSON, XML). Both local LLM inference frameworks (e.g., llamma_cpp, vllm) and online providers (e.g., OpenAI) provide an approach to limit the output tokens with a specific grammar.

## 3. Amalgam Algorithm

In this section, we present the algorithm Amalgam. First, we present the two main steps of the algorithm: structure learning (i.e., training the PGM and preparing the LLM), and sampling (i.e., generating synthetic data). Then, we present an evaluation method for determining the realism of synthetic data based on an assessment by an LLM agent.

### 3.1. Structure Learning

Structure learning begins by collecting metadata per entity in the dataset to form a tabular dataset distilling their analytics. An entity is defined as a single row in the main table of the dataset. For example, in a medical dataset, an entity may be a patient or an admission in the intensive care unit (ICU) if the dataset does not link to patients. In such cases, the main table is the patient table or the admission table, respectively. The analytics are defined as key statistics that we want to preserve in the synthetic data (e.g., the age distribution, treatment length, or medicine distribution).

To create the tabular dataset of analytics, first, the relational dataset is discretized to ensure all columns are categorical.

Numerical columns are binned into a set of discrete ranges. Integers are used, selecting the smallest unsigned integer width that fits the domain of the column (e.g., uint8 for values between 0 and 255). The core of the analytics table is formed by the main table of the dataset. Then, through a series of left joins, other tables in the dataset are merged into the main table, selecting one row per entity. If the relationship between the tables is sequential (e.g., admissions sorted by date), the first row is selected. Otherwise, an arbitrary row is chosen. Moreover, a count column is added per child table to indicate how many rows the original entity had.

The analytics tabular dataset is used to train a PGM model, e.g., PrivBayes (Zhang et al., 2017) or PrivMRF (Cai et al., 2021), which learns the joint distribution of the analytics. To ensure privacy when generating analytics, privacy-aware algorithms utilize the mechanism Differential Privacy (Dwork, 2006). By using a low privacy budget parameter $\epsilon$, e.g., 2, the algorithms ensure the analytics, which reference the whole dataset, are protected from inference. The resulting PGM is used to generate samples of the analytics, which will be used to condition the LLM during sampling. By avoiding the use of fine tuning or LLMs during structure learning, we avoid the privacy and performance implications these processes carry.

### 3.2. Sampling

During sampling, the PGM generates conditioning values (e.g., age, sex). These values are used with a similarity function to collect a number of top samples in the original data to be provided as a reference for synthesis. Then, the analytics are converted into a list of human readable values (e.g., "age: 45", "entry time: 10:30") and provided along-side the similar samples in a JSON format. These two components are used to form the prompt for the LLM, which is bound by a grammar to generate a JSON object that mirrors the dataset schema. The output of the LLM is parsed and returned as the synthetic sample.

**Similarity Function.** We define the similarity function as a function that receives as input the single column histograms of the analytics dataset, measured during the fitting of the PGM, and two samples, $a$, $b$. It can be formulated as follows:

$$similar(a, b) = \sum_{i=1}^{n} (a_i = b_i) \cdot \frac{1}{p_i} \tag{1}$$

Where $n$ is the number of columns, $a_i$ and $b_i$ are the values of column $i$ in samples $a$ and $b$ respectively, and $p_i$ is the probability of the value $a_i$ in the marginal of column $i$. An alternate representation of a single-column marginal is a histogram of the value counts of a column. This heuristic representation increases the perceived similarity between two samples proportionally to the number of values they have common and how rare these values are.

The similarity scores are generated per conditioning sample and row in the original data (complexity $O(MN)$, where $N$ is the number of rows in the original and $M$ is the number of conditioning samples). If $M = N$, this would result in a complexity of $O(N^2)$. However, as we show in the experimental section, the computational cost of each sampled row requires running the LLM, where the cost is $O(MF_{\text{LLM}})$, where $F_{\text{LLM}}$ is the cost of running the LLM per sample. As $F_{\text{LLM}}$ is significantly higher than $N$ similarity function calculations, $M << N$ must hold for synthesis to finish in a realistic timeframe. Therefore, the complexity of the similarity function is not a bottleneck in the overall synthesis process.

**LLM Synthesis.** First, the original dataset schema is converted into a JSON schema. This is done by mapping each row of the dataset to a dictionary, and modelling dependencies between tables as lists of those dictionaries. The conversion is performed by parsing the dataset schema recursively. Then, the $n$ samples are converted into this schema, by using human-readable values where possible (e.g., time value "5" becomes "4:00").

The samples and the conditioning values are used to form the prompt for the LLM as follows:

(**1**): **Identifier to the LLM:** The LLM is told it is a domain expert (e.g., a doctor for medical data) and that it is provided $n$ reference samples.

(**2**): **Reference Samples:** Then, the $n$ reference samples are provided in JSON format.

(**3**): **Conditioning Values:** Next, the conditioning values are provided in human-readable format (e.g., "age: 45", "entry time: 10:30").

(**4**): **Generation Instruction:** The LLM is instructed to generate a sample in JSON format that is similar to the conditioning values and reference samples.

(**5**): **Guiding Instructions:** Finally, if necessary, the LLM can be given a set of instructions to improve its output (e.g., "If the generated patient would be unrealistic, adjust the values slightly").

The structure of the prompt is designed to minimize context contamination while aiding the model to generate similar data. While providing reference samples is not required, not doing so could cause the synthetic output to not be similar to real data since the model would miss queues that come from observing the real data. Then, by placing the reference samples before the conditioning values and having the instructions last, we ensure that the model focuses on the conditioning values and remembers the guiding instructions

clearly. A different approach could optimize token caching by placing the guiding instructions first, and then the context, lowering the computational cost. We delegate the evaluation of such an approach to future work.

The LLM is then bound by the JSON schema of the dataset to ensure the output data is parseable through using a grammar-based approach (Ugare et al.; Tuccio et al., 2025). This ensures that the output is always valid JSON, which is especially important for complex schemas and when using local small models. Finally, the output JSON is parsed and converted to the relational format of the original data.

### 3.3. Realism Evaluation

A challenge that was faced by this work is the evaluation of the realism of synthetic data. As realism is a subjective metric, it is hard to quantify mechanically, via e.g., statistical metrics. For this reason, we introduce an unattended evaluation method for realism based on an LLM agent.

This method re-uses the core aspects of Amalgam's sampling process. Given a sample, the top $n$ similar samples are collected from the original dataset using the similarity function defined in Section 3.2. Then, all $n + 1$ samples (the synthetic and the $n$ original) are converted into JSON. Next, a prompt is formed for the LLM as follows:

(1): **Identifier to the LLM:** The LLM is told it is a domain expert and that it is provided $n$ *real* samples.

(2): **Real Samples:** The $n$ samples are provided as JSON.

(3): **Task:** The model is told to evaluate the realism of an additional sample provided to it and score it from 1 to 5 for realism.

(4): **Evaluation Sample:** Finally, the evaluated sample is provided to the LLM.

The LLM is expected to return a score from 1 to 5 and bound by grammar to ensure the output is parsable. The score is collected and averaged across multiple samples to provide an overall realism score for the synthetic dataset. Then, the evaluation is re-run using a hold-out set of real data samples that were not used for synthesis or eligible for selection as similar samples. This provides a realism score for real data, which can be used as a baseline for the synthetic data.

## 4. Experiments

We compare Amalgam against state-of-the-art in two experiments. First, we compare Amalgam against MARE (Kapenekakis et al., 2024), a PGM relational data synthesis algorithm, on four datasets. Then, we compare Amalgam's performance when using different local LLMs on one of the datasets.

### 4.1. Comparison with State-of-the-Art

In the first experiment, we compare Amalgam against the state-of-the-art algorithm MARE (Kapenekakis et al., 2024), which is an orchestration algorithm for relational data using multiple PGMs, according to which each relationship is classified as independent, associative, or sequential. We compare to MARE across four datasets: MIMIC-IV Admissions, eICU R1, CTUR SL, and CTUR CE.

MIMIC-IV Admissions is a dataset that consists of three tables in the core set of MIMIC-IV (Johnson et al.): "patients", "admissions", and "transfers". The tables "admissions" and "transfers" have a sequential relationship, where "transfers" is a child of "admissions". eICU R1 is a dataset that consists of four tables in the eICU dataset (Pollard et al., 2019): "patients", "admissiondx", "vitalaperiodic", and "medication", with an assortment of columns. All tables have a child relationship to "patients". The datasets CTUR SL and CTUR CE are sourced from The CTU Prague Relational Learning Repository (Motl & Schulte, 2024), where SL is the Student-Loans dataset, and CE is the ConsumerExpenditures dataset. CTUR SL contains key-only tables that show an attribute for a student (e.g., "disabled"). We merge those tables to the main "person" table to form a three table dataset with "person", "enrolled", and "enlist", where "enrolled" and "enlist" are associative child tables of "person". For CTUR CE, the tables "household", "person", and "expenditure" form a three table dataset. A dataset overview is shown in Table 1. It lists the number of tables, total number of columns, total number of values (i.e., rows multiplied by columns per table), number of entities (i.e., rows in the main table), and the average ratio of values to entities.

| Dataset | Tbls | Cols | Vals | N | Vals/N |
|---|---|---|---|---|---|
| MIMIC Adm. | 3 | 25 | 16.6M | 180.7k | 91.6 |
| eICU R1 | 4 | 25 | 141.3M | 200.9k | 703.5 |
| CTUR CE | 3 | 20 | 13.3M | 56.8k | 234.5 |
| CTUR SL | 3 | 10 | 9.2k | 1.0k | 9.2 |

*Table 1.* Dataset summary statistics.

In the experiments, we use the same PGM as MARE, PrivBayes (Zhang et al., 2017), with a total privacy budget of $\epsilon = 2$. We retain a 20% hold-out set for accuracy and realism evaluation. For accuracy, we collect an aggregated Kullback-Leibler (KL) divergence metric as shown in MARE (Kapenekakis et al., 2024) and a $\chi^2$ metric. Then, for realism, we use our proposed LLM-based evaluation method. We use Qwen3 8B (Yang et al., 2025) as the LLM for both synthesis and realism evaluation.

For the aggregated KL metric, we collect column pairs in three categories: intra-table pairs (both columns from the same table), inter-table pairs (columns from different tables that have a foreign key relationship), and sequential inter-

table pairs (with up to two previous rows). For each pair in the dataset, we calculate KL divergence between the original and synthetic (or real) data for each table and normalize them with the formula $f(x) = 1/(1 + x)$, which maps $[0, \infty)$ to $(0, 1]$. Then, we average the results per category, per-table, and finally we average the three categories to form the aggregated KL metric. For the $\chi^2$ metric, we collect the $\chi^2 P$ value per column between the original and synthetic (or real) data, average them per-table, and finally average the per-table results to form the final $\chi^2$ metric. $\chi^2 P$ values indicate the likelihood that two distributions were drawn from the same underlying distribution, with 100% being identical distributions.

Due to the higher computational demand of Amalgam and realism evaluation, we limit the number of samples that are generated and evaluated for realism to 2000 per dataset (excl. CTUR SL where it is 800 total). MARE always generates the same number of samples as the original dataset, and accuracy metrics (KL, $\chi^2$) are computed on all produced samples. The results are shown in Figure 1. Finally, we collect the synthesis time of Amalgam versus MARE for two steps: structure learning and sampling, shown in Table 2.

| | MARE | | Amalgam | |
|---|---|---|---|---|
| Dataset | Fit | Sample | Fit | Sample |
| MIMIC Adm. | 1m29s | 3s | 1m1s | 8h26m |
| eICU R1 | 4m7s | 3m33s | 25s | 12h30m |
| CTUR CE | 14.8s | 1s | 13s | 6h49m |
| CTUR SL | 11s | 0.3s | 4s | 7m7s |

*Table 2.* Time comparison between MARE and Amalgam.

### 4.2. LLM Comparison and Efficiency

In the second experiment, we compare Amalgam using four different LLMs for synthesis: Qwen3 8B (Yang et al., 2025), Gemma3 12B (GemmaTeam, 2025), GPT-OSS 20B (Agarwal et al., 2025), and Meta Llama 3.1 8B Instruct (Grattafiori et al., 2024). Qwen3 8B is used to evaluate realism in all four cases. We generate 250 samples. We use the MIMIC-IV Admissions dataset for this experiment and collect the same metrics as before. Then, we collect per-second energy and utilization data from the GPU, running the experiments back to back and recording the relative experiment time (wall time). The metrics results are shown in Figure 2 and the energy analysis in Figure 3.

### 4.3. Hardware Setup

The experiments are executed on units of Nvidia DGX Spark Founder Edition that contain a GB10, 128GB of RAM, and 4TB of SSD, using their stock operating system that mirrors Ubuntu 24.04, with Python 3.12. PGM models use the CPU and LLM inference the GPU. For each experiment, we collect per-second energy data from the GPU using "nvidia-smi" and the timing intervals of each synthesis step.

### 4.4. Implementation

We implement Amalgam by extending the data synthesis system Pasteur (Kapenekakis et al., 2026), which is written in Python. Pasteur provides the reference implementation of the algorithm MARE (Kapenekakis et al., 2024) using PrivBayes (Zhang et al., 2017), which is our state-of-the-art reference model. We implement the structure learning phase by re-using the fitting process of PrivBayes from Pasteur. Then, we implement the sampling phase by integrating the LLM inference framework "Llama.cpp" under the library "llama-cpp-python" and use the library "outlines" (Willard & Louf, 2023) to perform grammar-based generation.

We attempted to use online inference providers through OpenRouter but found that the structured generation via grammar failed when provided complex schemas for synthesis (e.g., the provider Groq only performed post-inference validation and discarded more than half of the samples) and had varying support per provider. The library "outlines" was also pushed to its limits, requiring around 1–3 minutes to prime the grammar for complex schemas and having around a 30% GPU throughput degradation compared to normal generation (shown in Figure 3 power drop).

We source local open-source models from HuggingFace (Wolf et al., 2020). Specifically, we opt to use the model Qwen3 8B (Yang et al., 2025) quantized to 4-bit ("Qwen/Qwen3-8B-GGUF", filename "Qwen3-8B-Q4_K_M.gguf") for its balance of quality and performance. For comparison, we compare it with models Gemma3 12B (GemmaTeam, 2025) ("unsloth/gemma-3-12b-it-GGUF", filename "gemma-3-12b-it-Q4_K_M.gguf"), GPT-Oss 20B (Agarwal et al., 2025) ("unsloth/gpt-oss-20b-GGUF", filename "gpt-oss-20b-Q4_K_M.gguf"), and Meta Llama 3.1 8B Instruct (Grattafiori et al., 2024) ("MaziyarPanahi/Meta-Llama-3.1-8B-Instruct-GGUF", filename "Meta-Llama-3.1-8B-Instruct.Q4_K_M.gguf"). At the time of this writing, only Qwen3 8B provided an official 4-bit GGUF quantization, so for the other models we used popular community provided quantizations.

## 5. Results

In the first experiment (Figure 1), we observe that Amalgam provides significantly higher realism than MARE across all datasets, closing the gap to real data. Overall, the average realism score across datasets for Amalgam is 3.8/5 compared to MARE's 3.3/5 and real data's 4.4/5. This is expected, as the LLM is able to leverage its foundational knowledge to generate more lifelike samples. Specifically, on MIMIC Admissions and CTUR SL, the Amalgam score distribution

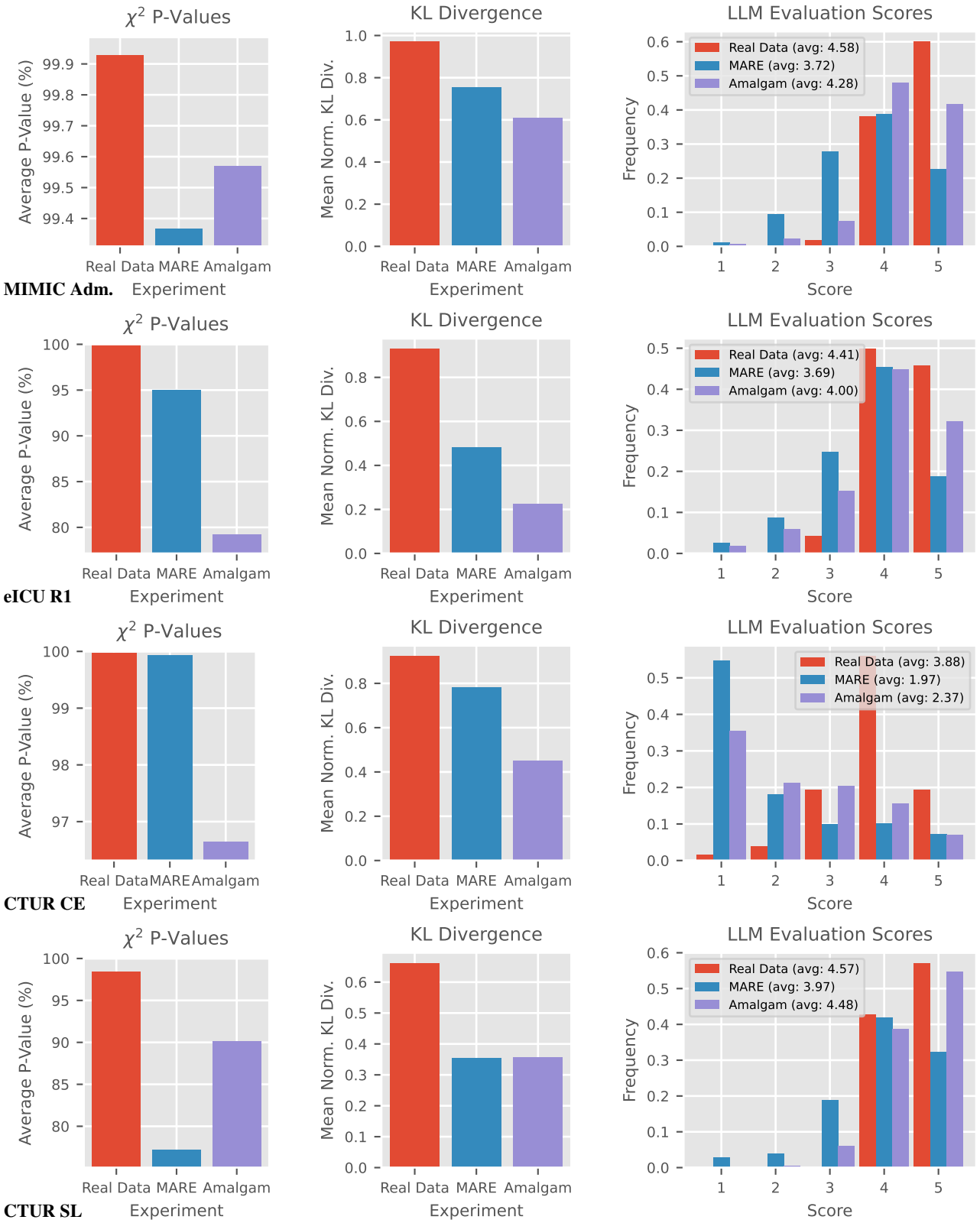


*Figure 1.* Comparison of Amalgam synthesis results across datasets versus MARE, Real Data.

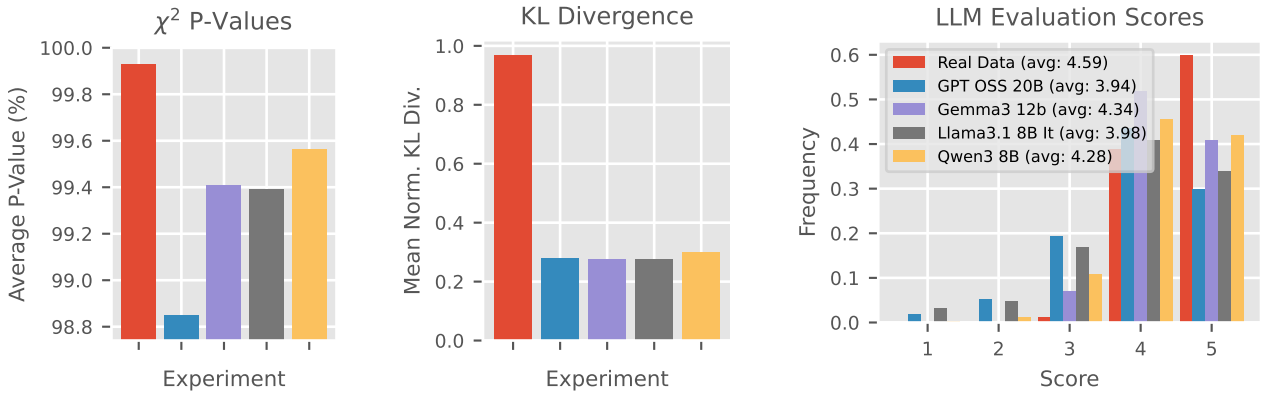


*Figure 2.* Comparison of Amalgam synthesis for different LLMs on MIMIC-IV Admissions.

from 1 to 5 overlaps significantly with real data, showing that the synthetic data is hard to distinguish from real data. On CTUR CE, the realism is improved significantly over MARE, but there is a significant number of unrealistic samples for both MARE and Amalgam. This is likely an effect of the dataset, which uses a lot of arbitrary product keys under "expenditures" that have low semantic meaning.

On accuracy, Amalgam is grounded in reality, with $\chi^2$ scores very close to 100%, showing high histogram quality. The exception is eICU R1, where the very large number of rows per entity causes the LLM to struggle to generate accurate counts for child tables. The histogram quality was additionally verified manually by observing histogram plots of the produced columns. The KL divergence scores are lower, showing that the joint distributions between columns are not as well preserved. As the LLM only has access to first row statistics (Section 3.1) and not follow-up row statistics, which would be required to preserve joint distributions, this is expected. Overall, the $\chi^2$ score across datasets for Amalgam is 91%, compared to MARE's 92%, and for KL divergence, Amalgam scores 0.40 with MARE at 0.60.

Reviewing the times on Table 2, we observe that Amalgam's structure learning is slightly faster than MARE's, as Amalgam only fits a single PGM. However, the sampling time is significantly higher, as sampling uses an LLM.

In the second experiment (Figure 2), we observe that Qwen3 8B and Gemma3 12B provide the best realism scores, with GPT-OSS and Meta Llama 3.1 8B Instruct lagging behind. On accuracy, all models provide similar $\chi^2$ scores, with Qwen3 8B having a slight edge. Due to the lower number of samples (250), the KL divergence score is negatively affected compared to the previous experiment, as rare combinations of values may not appear in the synthetic data. On the energy axis, we observe that Qwen3 8B uses significantly less energy than Gemma3 12B while producing slightly better results (Figure 3), making it the best choice for synthesis. GPT-OSS is surprisingly efficient, using two fifths of the energy and runtime of Qwen3 8B through its Mixture-of-Experts architecture with 3B parameters, but it performs worse. MARE runs on CPU only and only requires 1–5m to generate as many samples as in the original data, so its energy usage is negligible compared to LLMs. All models, including real data, are evaluated using Qwen3 8B for realism, so energy values for evaluation are the same across all models and real data (hold-out set).

## 6. Discussion

The results show that Amalgam is able to generate synthetic data that is significantly more realistic than PGM-based synthesis while preserving high accuracy. This is achieved by leveraging the foundational knowledge of LLMs to generate lifelike samples, while using PGMs to ground the generation in reality. Through combining these approaches, Amalgam handles complex datasets that PGM-based synthesis struggles to be realistic with, while avoiding the privacy and performance issues of LLM fine-tuning.

As Amalgam uses Differential Privacy in the PGM, it ensures that each synthetic sample is traceable to a fixed number of original samples, which can be used to audit the synthetic data for privacy violations. This is a significant advantage over pure LLM-based synthesis with fine-tuning, where the lack of traceability can lead to privacy concerns. Methods for automated screening of LLM outputs for privacy violations is a natural next step for future work.

Moreover, Amalgam runs comfortably on-premises, with all the experiments in this paper being performed on edge devices (DGX Sparks) with acceptable runtimes ($\sim$ 7 hours for 2000 samples), while representing the whole dataset, which is an excellent attribute for sensitive data synthesis. Due to its PGM foundation, Amalgam can scale to very large datasets and fit them in minutes, with its generated

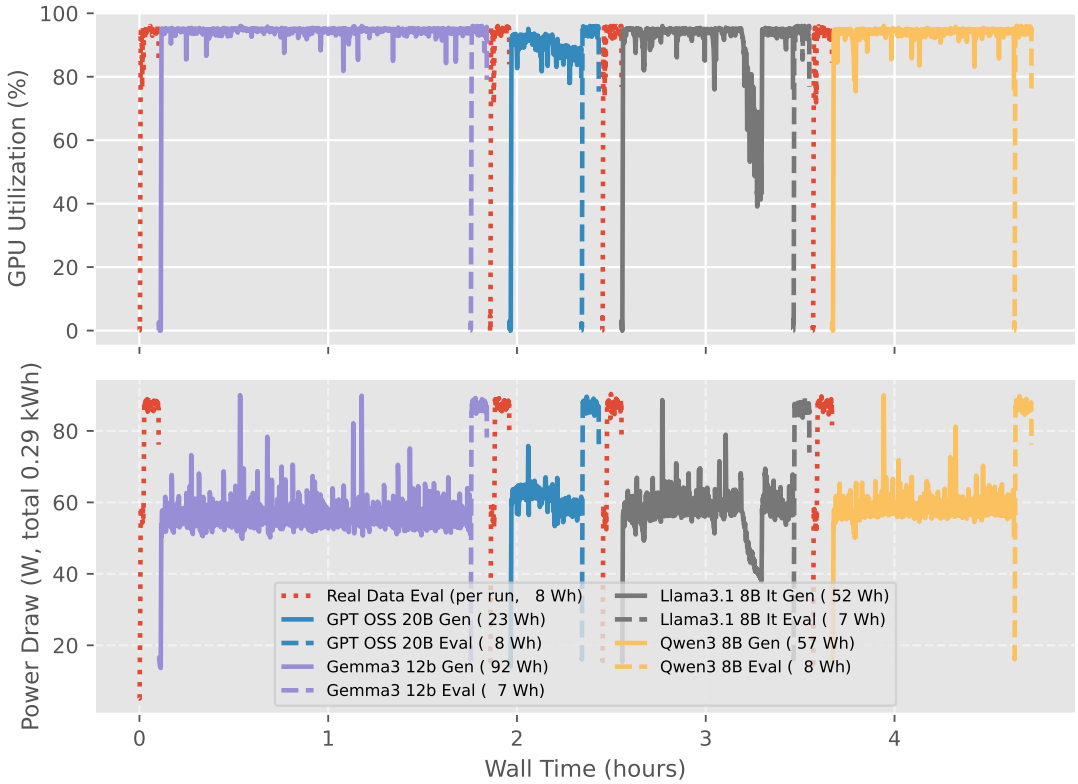


*Figure 3.* Energy usage comparison across different LLMs on MIMIC-IV Admissions dataset.

data being representative of the entire input dataset and, if on-demand generation is needed, Amalgam can begin generating synthetic data a few minutes after cold-start.

However, Amalgam has limitations in certain use-cases where PGMs would be preferable. For example, if the goal is purely analytical and the data will not be used for human consumption, PGM-based synthesis can produce comparable analytics while being significantly faster and more energy efficient. In that regard, if the data will be used to stress test systems (e.g., databases), PGM-based synthesis is a better fit.

In summary, Amalgam has a complementary use-case to PGM-based synthesis, being preferable when the data will be used for human consumption (e.g., application testing) and a large amount of output data is not required. When analytics are the main goal and large amounts of data are required, PGM-based synthesis remains preferable. As an algorithm, Amalgam's resource requirements are relatively low, allowing it to run on-premises or on edge devices. In sum, we envision that Amalgam can be used by organizations in combination with a PGM algorithm, where Amalgam is used to generate a small set of realistic samples (in the thousands) for UI stand-ins and PGM synthesis to generate large amounts of accurate samples (in the hundreds of millions) for e.g., database stress-testing.

## 7. Conclusion

In this paper, we presented Amalgam, a hybrid LLM-PGM synthesis algorithm that leverages the strengths of LLMs and PGMs to generate lifelike synthetic data. Amalgam uses PGMs to learn the joint distribution of key analytics in the dataset, ensuring that the synthetic data is grounded in reality, while using pre-trained LLMs to generate samples that are similar to real data. This combination allows Amalgam to handle complex datasets that PGMs struggle with, while avoiding the privacy and performance issues of LLM fine-tuning. Through experiments on multiple datasets, we show that Amalgam produces synthetic data that is significantly more realistic than PGM-based synthesis while preserving high accuracy. Moreover, Amalgam runs comfortably on-premises, making it suitable for sensitive data synthesis.

## A. Acknowledgments

This work was supported by the VILLUM Foundation under project "Teaching AI Green Coding" (VIL70090); by ITEA4 and the Innovation Fund Denmark for projects "GreenCode" (2306) and "MAST" (22035).

## B. LLM Prompts

In this appendix, we provide the prompts used for LLM synthesis for each dataset, for synthesis and realism evaluation. In code, the placeholders `<samples_n>`, `<samples>`, `<seed>`, and `<eval>` are replaced with the number of samples, the reference samples in JSON format, the conditioning values in human-readable format, and the evaluated sample in JSON format respectively.

### B.1. MIMIC-IV Admissions

#### Synthesis Prompt

```
You are a doctor working at a
hospital.

Reference the following <samples_n>
example patients:
<samples>

And derive a JSON patient using:
<seed>

Guidelines:

• the provided values are for the
  first admission/transfer, generate
  the rest in a realistic manner
• for provided ranges use a random
  value that is within the range
• Reason about the key events in
  this patient's admission; they
  should make sense
• If a value would cause an
  unrealistic patient, adjust it
  slightly for realism
```

#### Evaluation Prompt

```
You are a medical doctor.

You are given the following
<samples_n> real patients as a
reference:
<samples>

Then, you are asked to comment on
how real the following patient is
with a rating from 1 to 5 (5 being
very real):
<eval>
```

### B.2. eICU R1

#### Synthesis Prompt

```
You are a doctor working at a
hospital.

Reference the following <samples_n>
example patients:
<samples>

And derive a JSON patient using:
<seed>

Guidelines:

• the provided values are for the
  first admissions etc, generate the
  rest in a realistic manner
• for provided ranges use a random
  value that is within the range
• Reason about the key events in
  this patient's admission; they
  should make sense
• If a value would cause an
  unrealistic patient, adjust it
  slightly for realism
```

#### Evaluation Prompt

```
You are a medical doctor.

You are given the following
<samples_n> real patients as a
reference:
<samples>

Then, you are asked to comment on
how real the following patient is
with a rating from 1 to 5 (5 being
very real):
<eval>
```

### B.3. CTUR CE

#### Synthesis Prompt

```
You are a tax accountant.

Reference the following <samples_n>
households:
<samples>

And derive a new JSON household
using:
<seed>

Guidelines:

• the provided values are for the
  first expense/member, generate the
  rest in a realistic manner
• for provided ranges use a random
  value that is within the range and
  4 decimal places
• If a value would cause an
  unrealistic household, adjust it
  slightly for realism
```

#### Evaluation Prompt

```
You are a tax accountant.

You are given the following
<samples_n> real households as a
reference:
<samples>

Then, you are asked to comment on
how real the following household is
with a rating from 1 to 5 (5 being
very real):
<eval>
```

### B.4. CTUR SL

#### Synthesis Prompt

```
You are a school administrator.
Reference the following <samples_n>
student profiles:
<samples>
And derive a new JSON student using:
<seed>
Guidelines:
• the provided values are for the
  first enrollment/enlistment,
  generate the rest in a realistic
  manner
• for provided ranges use a random
  value that is within the range and
  4 decimal places
• If a value would cause an
  unrealistic student, adjust it
  slightly for realism
```

#### Evaluation Prompt

```
You are a school administrator.
You are given the following
<samples_n> real students as a
reference:
<samples>
Then, you are asked to comment on
how real the following student is
with a rating from 1 to 5 (5 being
very real):
<eval>
```

## References


Agarwal, S., Ahmad, L., Ai, J., Altman, S., et al. gpt-oss-120b & gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Cai, K., Wei, J., Lei, X., and Xiao, X. Data Synthesis via Differentially Private Markov Random Fields. *VLDB*, 2021.

Cai, K., Xiao, X., and Cormode, G. Privlava: Synthesizing relational data with foreign keys under differential privacy. *Proc. ACM Manag. Data*, 1(2):142:1–142:25, 2023.

Chan, Y.-C., Pu, G., Shanker, A., Suresh, P., Jenks, P., Heyer, J., and Denton, S. Balancing Cost and Effectiveness of Synthetic Data Generation Strategies for LLMs.

Dwork, C. Differential privacy. 2006.

et al., D.-A. Deepseek-v3 technical report. 12 2024. URL https://arxiv.org/pdf/2412.19437.

GemmaTeam. Gemma 3 technical report. 3 2025. URL https://arxiv.org/pdf/2503.19786.

Goyal, M. and Mahmoud, Q. H. An LLM-Based Framework for Synthetic Data Generation. *2025 IEEE 15th Annual Computing and Communication Workshop and Conference, CCWC 2025*, pp. 340–346, 2025. doi: 10.1109/CCWC62904.2025.10903878.

Grattafiori, A., Dubey, A., Jauhri, A., et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Guo, K., Chen, J., Qiu, T., Guo, S., Luo, T., Chen, T., and Ren, S. Medgan: An adaptive GAN approach for medical image generation. *Comput. Biol. Medicine*, 163:107119, 2023.

Johnson, A., Bulgarelli, L., Pollard, T., Horng, S., Celi, L. A., and Mark, R. MIMIC-IV.

Jordon, J., Yoon, J., and Van Der Schaar, M. Pate-gan: Generating synthetic data with differential privacy guarantees. In *International conference on learning representations*, 2018.

Kapenekakis, A., Dell'aglio, D., Vesteghem, C., Poulsen, L., Bøgsted, M., Garofalakis, M., and Hose, K. Synthesizing accurate relational data under differential privacy. *IEEE BigData 2024*, 2024.

Kapenekakis, A., Dell'Aglio, D., Bøgsted, M., Garofalakis, M., and Hose, K. Pasteur: Scaling privacy-aware data synthesis. In Chrysanthis, P. K., Nørvåg, K., Stefanidis, K., and Zhang, Z. (eds.), *Advances in Databases and Information Systems*, pp. 164–180, Cham, 2026. Springer Nature Switzerland. ISBN 978-3-032-05281-0.

Koller, D. and Friedman, N. Structure learning in bayesian networks. *Probabilistic Graphical Models: Principles and Techniques*, pp. 783–848, 2009.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 5 2024. URL https://github.com/mit-han-lab/llm-awq.

Mckenna, R., Sheldon, D., and Miklau, G. Graphical-model based estimation and inference for differential privacy. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 4435–4444. PMLR, May 2019.

McKenna, R., Miklau, G., and Sheldon, D. Winning the NIST Contest: A scalable and general approach to differentially private synthetic data, August 2021.

McKenna, R., Mullins, B., Sheldon, D., and Miklau, G. AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data, January 2022.

Motl, J. and Schulte, O. The ctu prague relational learning repository, 2024. URL https://arxiv.org/abs/1511.03086.

Pollard, T., Johnson, A., Raffa, J., Celi, L. A., Badawi, O., and Mark, R. eICU Collaborative Research Database, April 2019. URL https://doi.org/10.13026/C2WM1R. Version 2.0.

Song, S., Chaudhuri, K., and Sarwate, A. D. Stochastic gradient descent with differentially private updates. In *2013 IEEE Global Conference on Signal and Information Processing*, 2013.

Tao, Y., McKenna, R., Hay, M., Machanavajjhala, A., and Miklau, G. Benchmarking Differentially Private Synthetic Data Generation Algorithms. December 2021.

Tuccio, G., Bulla, L., Madonia, M., Gangemi, A., and Mongiovi', M. Grammar-llm: Grammar-constrained natural language generation. pp. 3412–3422, 8 2025. doi: 10.18653/V1/2025.FINDINGS-ACL.177. URL https://aclanthology.org/2025.findings-acl.177/.

Ugare, S., Suresh, T., Kang, H., Misailovic, S., and Singh, G. Syncode: Llm generation with grammar augmentation. *Transactions on Machine Learning Research*. ISSN 2835-8856. URL https://github.com/uiuc-focal-lab/syncode.

Vaswani, A., Brain, G., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Łukasz Kaiser, and Polosukhin, I. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Willard, B. T. and Louf, R. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2020. doi: 10.48550/arXiv.1910.03771. URL https://arxiv.org/abs/1910.03771.

Xu, L., Skoularidou, M., Cuesta-Infante, A., and Veeramachaneni, K. Modeling Tabular data using Conditional GAN. 2019.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Yang, D., Monaikul, N., Ding, A., Tan, B., Mosaliganti, K., and Iyengar, G. Enhancing Table Representations with LLM-powered Synthetic Data Generation. nov 2024. URL https://arxiv.org/pdf/2411.03356.

Zhang, J., Cormode, G., Procopiuc, C. M., Srivastava, D., and Xiao, X. PrivBayes: Private Data Release via Bayesian Networks. *ACM Transactions on Database Systems*, 42(4):25:1–25:41, October 2017.